R Notebook

Figure 1

```
n <- 100
tmax <- m <- 30
p <- 0.3
q <- 0.9
delta <- (1-0.1)/tmax
tstar <- m/2
```

```
True_D <- true_London_dMV(tmax,tstar,p,q,delta)
MDS_True_D <- doMDS(True_D, doplot = F)
True_shuffled_D = sqrt(true_shuffled_dMV_square_London(tmax,tstar,p,q,delta))

MDS_True_shuffled_D = doMDS(True_shuffled_D , doplot = F)
```

```
set.seed(5)
df <- doSim_London(n, tmax, delta, p, q, tstar)
```

```
del = c(0.01,0.05,0.1,0.2,0.3,0.4,0.5,1)

for (alpha in del) {
  nm <- paste0("True_shuffle_dmv_", alpha)
  assign(nm,
         sqrt((1 - alpha) * True_D^2 + alpha * True_shuffled_D^2),
         envir = .GlobalEnv)
}

MDS_True_shuffle_0.2 <- doMDS(True_shuffle_dmv_0.2, doplot = F)

for(perc in del){
  df <- df %>%
    mutate(!!paste0("shuffle_Xt_", perc) := map( Xt ,~ shuffle_X(., perc)) ) %>%
    mutate(!!paste0("shuffle_Xhat_", perc) := map( xhat ,~shuffle_X(., perc)) )
}


D2_shuffle_0.1_hat <- getD(df$shuffle_Xhat_0.1)
MDS_shuffle_0.1_hat <- doMDS(D2_shuffle_0.1_hat, doplot = F)

D2_shuffle_0.2_hat <- getD(df$shuffle_Xhat_0.2)
MDS_shuffle_0.2_hat <- doMDS(D2_shuffle_0.2_hat, doplot = F)

D2_shuffle_1 <- getD(df$shuffle_Xhat_1)
mds_hat_shuffled_dMV = doMDS(D2_shuffle_1, doplot= F)

HatdMV_D <- getD(df$xhat)
mds_hat_dMV <- doMDS(HatdMV_D,doplot = F)
```

```
D_london_W1 = getD_W1(df$xhat)
London_W1_mds = doMDS(D_london_W1, doplot = F)
true_London_W1_mds = doMDS(sqrt(true_W1_square_London(tmax, tstar , p , q ,delta)), doplot=F)
#par(mfrow= c(1,3))
```

```
df_plot <- bind_rows(
  # dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_D$mds[,1],
         metric = "dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = mds_hat_dMV$mds[,1],
         metric = "dMV",
         type   = "estimated"),
  tibble (x = 1:tmax/tmax,
          y = (psi_Z(1:tmax/tmax,p,q)- mean(psi_Z(1:tmax/tmax,p,q)))*0.9,
          metric = "dMV",
          type = 'psi z'
  ),

  # 20% shuffled-dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_shuffle_0.2$mds[,1],
         metric = "20%-shuffled-dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = MDS_shuffle_0.2_hat$mds[,1],
         metric = "20%-shuffled-dMV",
         type   = "estimated"),
  tibble (x = 1:tmax/tmax,
          y = (psi_Z(1:tmax/tmax,p,q)- mean(psi_Z(1:tmax/tmax,p,q)))*0.9,
          metric = "20%-shuffled-dMV",
          type = 'psi z'
  ),

  # 100% shuffled-dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_shuffled_D$mds[,1],
         metric = "100%-shuffled-dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = mds_hat_shuffled_dMV$mds[,1],
         metric = "100%-shuffled-dMV",
         type   = "estimated"),
  tibble (x = 1:tmax/tmax,
          y = (psi_Z(1:tmax/tmax,p,q)- mean(psi_Z(1:tmax/tmax,p,q)))*0.9,
          metric = "100%-shuffled-dMV",
          type = 'psi z'
  ),
  # W1 distance
  tibble(x      = 1:tmax/tmax,
         y      = (psi_Z(1:tmax/tmax, p, q) - mean(psi_Z(1:tmax/tmax, p, q)))*0.9,
```

```
        metric='W1',
        type   = "psi z"),
  tibble(x      = 1:tmax/tmax,
        y      = true_London_W1_mds$mds[,1],
        metric='W1',
        type   = "true"),
  tibble(x      = 1:tmax/tmax,
        y      = London_W1_mds$mds[,1],
        metric='W1',
        type   = "estimated"
  ),
  # avg degree
  tibble(x      = 1:tmax/tmax,
        y      = sqrt(true_avg_degree(1:tmax))/sqrt(n-1)-mean(sqrt(true_avg_degree(1:tmax))/sqrt(n-1))
        metric = "avg degree",
        type   = "true"),
  tibble(x      = 1:tmax/tmax,
        y      = sqrt(unlist(df$avg_edges) / (n) )/sqrt(n-1)- mean(sqrt(unlist(df$avg_edges) / (n) )/s
        metric = "avg degree",
        type   = "estimated"),
  tibble (x = 1:tmax/tmax,
         y = (psi_Z(1:tmax/tmax,p,q)- mean(psi_Z(1:tmax/tmax,p,q)))*0.9,
         metric = "avg degree",
         type = 'psi z')
) %>%
  mutate(metric = factor(
    metric,
    levels = c("dMV" ,"20%-shuffled-dMV","100%-shuffled-dMV",  "W1","avg degree"),
    labels = c(
      "d[MV]",
      "20*\"%\"~shuffled~d[MV]",
      "100*\"%\"~shuffled~d[MV]",
      "W[1]",
      "sqrt(plain('avg degree')/(n-1))"
    )
  ))

ggplot(df_plot, aes(x = x, y = y, color = type)) +
  geom_point(
    data = df_plot %>% filter(type %in% c("true","estimated")),
    aes(x = x, y = y, color = type),
    size  = 2,
    alpha = 0.5
  ) +
  geom_line(
    data = df_plot %>% filter(type == "psi z"),
    aes(x = x, y = y),
    color = "red",
    size  = 1
  )+
  facet_wrap(~ metric,
             nrow     = 1,
             scales   = "free_y",
```

```
            labeller = label_parsed) +
  labs(x = "time",
       y = "graph dynamics representation") +
  scale_color_manual(values = c(true = "black",
                                estimated = "blue")) +
  theme_bw() +
  theme(
    legend.position = "none",
    strip.text      = element_text(size = 12, face = "bold"),
    axis.title      = element_text(size = 14, face = "bold"),
    axis.text       = element_text(size = 12)
  )
```

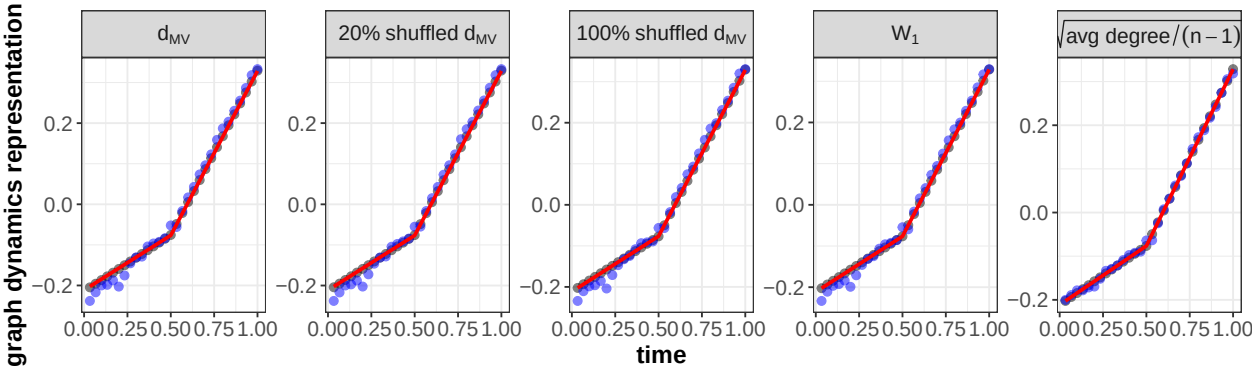


Figure 2 (without W1)

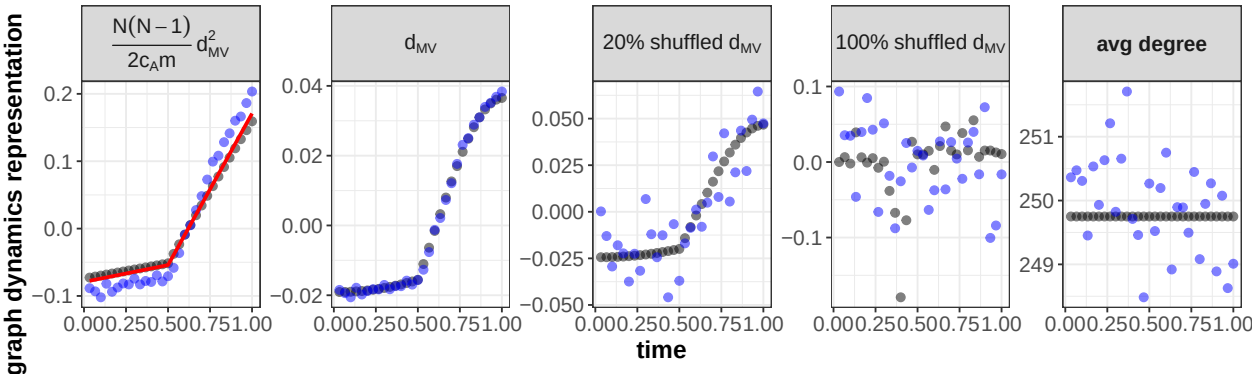
```
## Atlanta
p = 0.05
q = 0.45
tmax = m = 30
tstar = 15

c=(.9-0.1)
num_state = 50
delta = c/(num_state-1)

True_dmv_square=true_Atlanta_dmv(p,q,num_state,m,tstar,delta) ## This is the analytically d^2MV result.
True_dmv = sqrt(True_dmv_square)
MDS_True_D_square =doMDS(True_dmv_square, doplot = F)
MDS_True_D =doMDS(sqrt(True_dmv_square), doplot = F)


yy = MDS_True_D_square$mds[,1]*num_state*(num_state-1)/(2*c^2*m)

if( yy[1] > 0){ yy = -yy} else {
    yy = yy
  }


True_shuffle_dmv_square=true_shuffle_Atlanta_dmv(c, num_state , m )
True_shuffle_dmv = sqrt(True_shuffle_dmv_square)
MDS_True_shuffle_D = doMDS(True_shuffle_dmv, doplot = F)
```

```r
set.seed(2)
n = 1000
xt=matrix(0,nrow = n, ncol = m+1)
initila_state_all_nodes=sample(seq(.1,.9,by=delta),n,replace = TRUE)

the_var = c^2/12*( (num_state+1)/(num_state-1) )

xt[,1]=initila_state_all_nodes
for (i in 1:n) {
  for (j in 2:(tstar)) {
    xt[i,j]=update_function(xt[i,j-1],p,delta)
  }
  for (j in ((tstar+1):(m+1)) ) {
    xt[i,j]=update_function(xt[i,j-1],q,delta)
  }
}
df <- tibble(time=1:m) %>%
  mutate(Xt = map(time, function(x) matrix(xt[,x],n,1)  ))  %>%
  mutate(g = map(Xt, ~rdpg.sample(.))) %>%
  mutate(avg_edges = map( g ,  ~sum(as.matrix(.))    )   ) %>%
  mutate(xhat = map(g, function(x) full.ase(x,2)$Xhat[,1,drop=F]))

D2 <- getD(df$xhat)
MDS_hat_dMV_square = doMDS(D2^2, doplot= F)


MDS_hat_dMV = doMDS(D2, doplot= F)


del = c(0.01,0.05,0.1,0.2,0.3,0.4,0.5,1)

for (alpha in del) {
  nm <- paste0("True_shuffle_dmv_", alpha)
  assign(nm,
         sqrt((1 - alpha) * True_dmv_square + alpha * True_shuffle_dmv_square),
         envir = .GlobalEnv)
}

MDS_True_shuffle_0.2 <- doMDS(True_shuffle_dmv_0.2, doplot = F)

P=diag(rep(1,tmax))-rep(1,tmax)%*%t(rep(1,tmax))/tmax
B=(-1/(2))*P%*%True_dmv_square%*%P


for(perc in del){
  df <- df %>%
    mutate(!!paste0("shuffle_Xt_", perc) := map( Xt ,~ shuffle_X(., perc)) ) %>%
    mutate(!!paste0("shuffle_Xhat_", perc) := map( xhat ,~shuffle_X(., perc)) )
}


D2_shuffle_0.1_hat <- getD(df$shuffle_Xhat_0.1)
MDS_shuffle_0.1_hat <- doMDS(D2_shuffle_0.1_hat, doplot = F)
```

```r
D2_shuffle_0.2_hat <- getD(df$shuffle_Xhat_0.2)
MDS_shuffle_0.2_hat <- doMDS(D2_shuffle_0.2_hat, doplot = F)



D2_shuffle_1 <- getD(df$shuffle_Xhat_1)
MDS_hat_shuffle_dMV = doMDS(D2_shuffle_1, doplot= F)


df_plot <- bind_rows(
  # d^2MV
  tibble(x      = 1:tmax/tmax,
         y      = yy,
         metric = "d^2MV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = MDS_hat_dMV_square$mds[,1]*num_state*(num_state-1)/(2*c^2*m),
         metric = "d^2MV",
         type   = "estimated"),
  #dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_D$mds[,1],
         metric = "dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = MDS_hat_dMV$mds[,1],
         metric = "dMV",
         type   = "estimated"),
  # 20% shuffled-dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_shuffle_0.2$mds[,1],
         metric = "20%-shuffled-dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = MDS_shuffle_0.2_hat$mds[,1],
         metric = "20%-shuffled-dMV",
         type   = "estimated"),
  # 100%-shuffled-dMV
  tibble(x      = 1:tmax/tmax,
         y      = MDS_True_shuffle_D$mds[,1],
         metric = "100%-shuffled-dMV",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = MDS_hat_shuffle_dMV$mds[,1],
         metric = "100%-shuffled-dMV",
         type   = "estimated"),
  # avg deg
  tibble(x      = 1:tmax/tmax,
         y      = rep(0.5^2*(n-1),tmax),
         metric = "avg degree",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
```

```
        y      = unlist(df$avg_edges) / n,
        metric = "avg degree",
        type   = "estimated"),
  #psi Z
  tibble (x = 1:tmax/tmax,
          y = psi_Z(1:tmax/tmax,p,q)- mean(psi_Z(1:tmax/tmax,p,q)),
          metric = "d^2MV",
          type = 'psi z'
          )
) %>%
  mutate(metric = factor(
    metric,
    levels = c("d^2MV","dMV","20%-shuffled-dMV","100%-shuffled-dMV", "avg degree"),
    labels = c(
      "frac(N*(N-1), 2*c[A]*m) ~ d[MV]^2",
      "d[MV]",
      "20*\"%\"~shuffled~d[MV]",
      "100*\"%\"~shuffled~d[MV]",
      "'avg degree'"
    )
  ))
ggplot() +
  # 1) your black/blue points for "true" & "estimated"
  geom_point(
    data = df_plot %>% filter(type %in% c("true","estimated")),
    aes(x = x, y = y, color = type),
    size  = 2,
    alpha = 0.5
  ) +
  scale_color_manual(values = c(true = "black", estimated = "blue")) +

  # 2) overlay psi_Z as a red line in the d^2MV panel
  geom_line(
    data = df_plot %>% filter(type == "psi z"),
    aes(x = x, y = y),
    color = "red",
    size  = 1
  ) +

  # 3) faceting and labels
  facet_wrap(~ metric,
             nrow     = 1,
             scales   = "free_y",
             labeller = label_parsed) +
  labs(x = "time",
       y = "graph dynamics representation") +

  theme_bw() +
  theme(
    legend.position = "none",
    strip.text      = element_text(size = 12, face = "bold"),
    axis.title      = element_text(size = 14, face = "bold"),
    axis.text       = element_text(size = 12)
```

```
  )
```



Figures of W1 distance for both London and Atlanta

```
## W1 distance

set.seed(3)

n=1000
c=(.9-0.1)
p <- 0.05
q <- 0.45
num_state = 50
delta = c/(num_state-1)
xt=matrix(0,nrow = n, ncol = m+1)
initila_state_all_nodes=sample(seq(.1,.9,by=delta),n,replace = TRUE)

xt[,1]=initila_state_all_nodes
for (i in 1:n) {
  for (j in 2:(tstar)) {
    xt[i,j]=update_function(xt[i,j-1],p,delta)
  }
  for (j in ((tstar+1):(m+1)) ) {
    xt[i,j]=update_function(xt[i,j-1],q,delta)
  }
}
df <- tibble(time=1:m) %>%
  mutate(Xt = map(time, function(x) matrix(xt[,x],n,1)  ))  %>%
  mutate(g = map(Xt, ~rdpg.sample(.))) %>%
  mutate(avg_edges = map( g ,  ~sum(as.matrix(.))    )   ) %>%
  mutate(xhat = map(g, function(x) full.ase(x,2)$Xhat[,1,drop=F]))

D_atlanta_W1 = getD_W1(df$xhat)
atlanta_W1_mds = doMDS(D_atlanta_W1, doplot = F)


n = 100
tmax <- m <- 30
p <- 0.3
q <- 0.9
delta <- (1-0.1)/tmax
```

```r
tstar <- m/2

df <- doSim_London(n, tmax, delta, p, q, tstar)
df <- df %>%
  mutate(shuffle_g = map(g, ~optimized_shuffle_graph(.))) %>%
  mutate(Xhat_shuffle = map(shuffle_g, function(x) full.ase(x,2)$Xhat[,1,drop=F]))

D_london_W1 = getD_W1(df$xhat)
London_W1_mds = doMDS(D_london_W1, doplot = F)
true_London_W1_mds = doMDS(sqrt(true_W1_square_London(tmax, tstar , p , q,delta )), doplot=F)

df_plot <- bind_rows(
  # London
  tibble(x      = 1:tmax/tmax,
         y      = (psi_Z(1:tmax/tmax, p, q) - mean(psi_Z(1:tmax/tmax, p, q)))*0.9,
         metric='W1',
         model = "london",
         type   = "psi_Z"),
  tibble(x      = 1:tmax/tmax,
         y      = true_London_W1_mds$mds[,1],
         metric='W1',
         model = "london",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = London_W1_mds$mds[,1],
         model = "london",
         metric='W1',
         type   = "estimated"),
  # Atlanta
  tibble(x      = 1:tmax/tmax,
         y      = rep(0,tmax) ,
         model = "Atlanta",
         metric = "W1",
         type   = "true"),
  tibble(x      = 1:tmax/tmax,
         y      = atlanta_W1_mds$mds[,1],
         model = "Atlanta",
         metric = "W1",
         type   = "estimated")
)


# 1) ensure 'model' is a factor in the order you want
df_plot <- df_plot %>%
  mutate(
    model = factor(
      model,
      levels = c("london", "Atlanta"),
      labels = c("London", "Atlanta")
    )
  )

# 2) draw the two-panel plot, overriding each strip label to "W1"
```

```r
ggplot() +
  # black & blue points
  geom_point(
    data = df_plot %>% filter(type %in% c("true", "estimated")),
    aes(x = x, y = y, color = type),
    size  = 3,
    alpha = 0.6
  ) +
  scale_color_manual(values = c(true = "black", estimated = "blue")) +

  # red psi_Z line (only for London, since only there you have type=="psi_Z")
  geom_line(
    data = df_plot %>% filter(type == "psi_Z"),
    aes(x = x, y = y),
    color = "red",
    size  = 1
  ) +

  # facet into two side-by-side panels, but force both strip titles to "W1"
  facet_wrap(
    ~ model,
    nrow     = 1,
    scales   = "free_y",
    labeller = labeller(model = c(London = "W1 London", Atlanta = "W1 Atlanta"))
  ) +

  # shared axis labels
  labs(
    x = "time",
    y = "graph dynamics representation"
  ) +

  theme_bw() +
  theme(
    legend.position = "none",
    strip.text      = element_text(size = 12, face = "bold"),
    axis.title      = element_text(size = 14, face = "bold"),
    axis.text       = element_text(size = 12)
  )
```

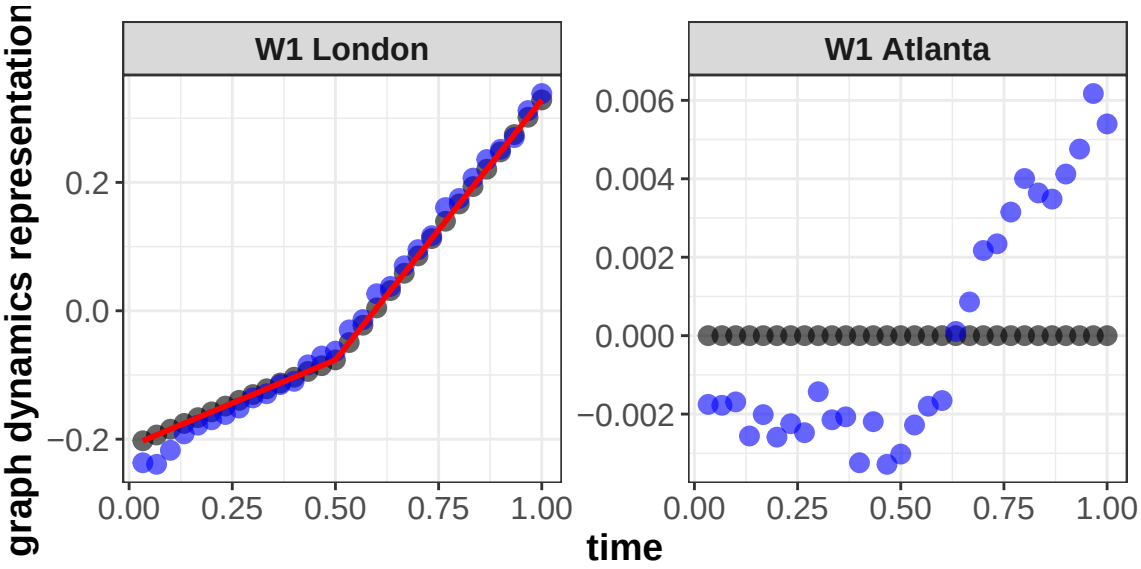
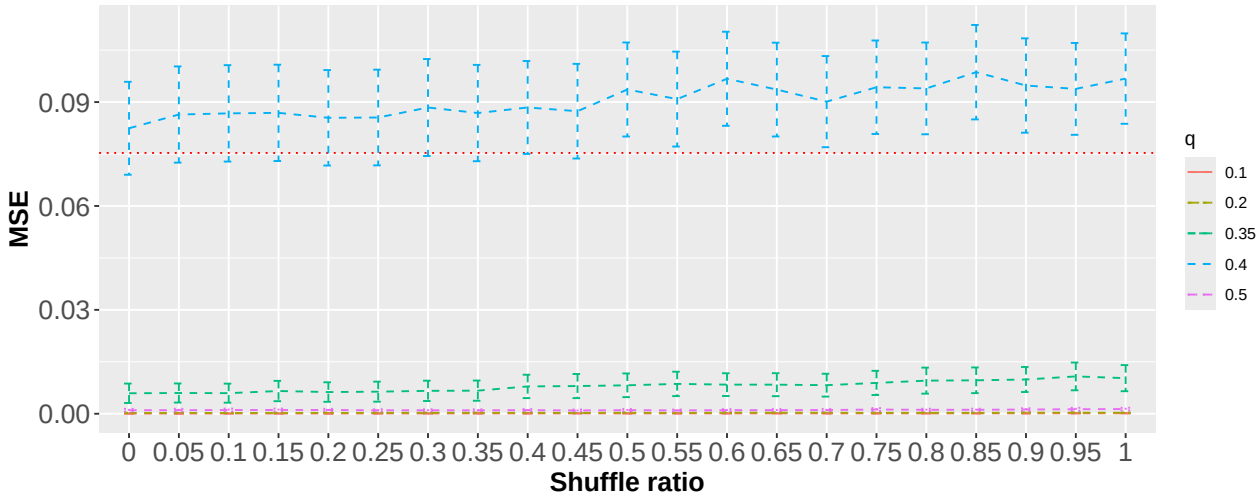


Figure 3

London

```
m <- 40
tstar <- 20
Num_states <- m
p <- 0.4
q <- c(0.1, 0.2, 0.35, 0.4, 0.5)
d <- seq(0.05, 1, by = 0.05)
n <- 300
nmc <- 100

library(readr)

final_errors <- read_csv("../simulation/simulation_results/final_errors_london.csv")
# This file includes results of the simulation on London model for both l2 and l-infinity localizers, w
final_mse=final_errors[,seq(1,20,4)]
final_sd=final_errors[,seq(2,20,4)]

support = seq(2/m-0.5, (m-1)/m-0.5,by=1/m)
chance_level = sum(support^2/length(support))


library(ggplot2)
library(reshape2)
# Convert matrix to data frame for ggplot2
data_melt <- melt(as.matrix(final_mse))
data_melt_sd <- melt(as.matrix(final_sd))

shuffle_ratio=c(0,d)
data_melt$Var1 <- factor(data_melt$Var1, labels = shuffle_ratio)
data_melt$Var2 <- factor(data_melt$Var2, labels = q)

data_melt_sd$Var1 <- factor(data_melt_sd$Var1, labels = shuffle_ratio)
```

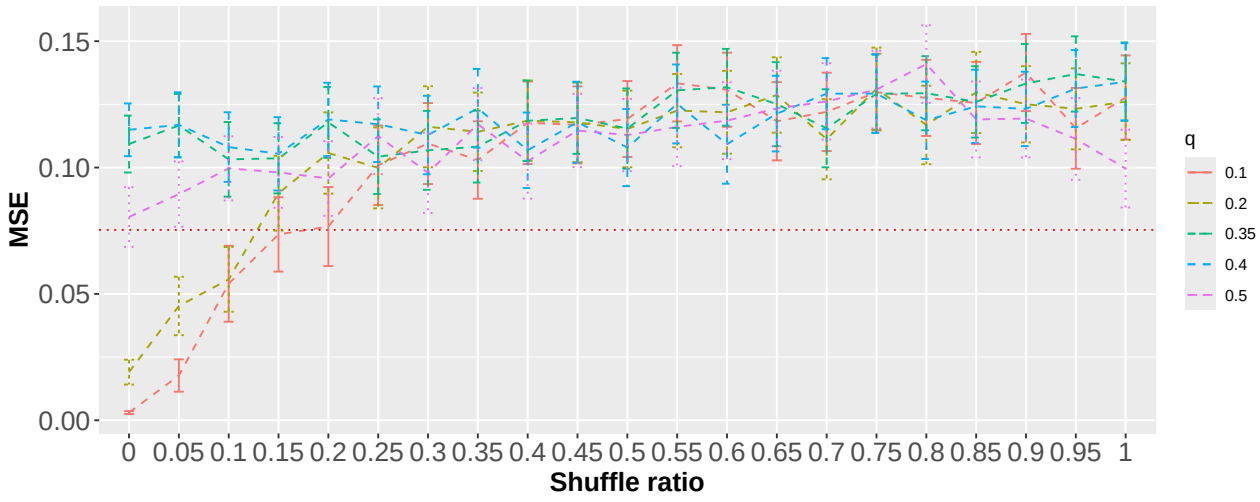
```r
data_melt_sd$Var2 <- factor(data_melt_sd$Var2, labels = q)


colnames(data_melt)=c('shuffle ratio','q','mean')
colnames(data_melt_sd)=c('shuffle ratio','q','sd')

summary=cbind(data_melt,data_melt_sd$sd)

summary$lower=summary$mean-summary$`data_melt_sd$sd`*1.96
summary$upper=summary$mean+summary$`data_melt_sd$sd`*1.96


library(dplyr)  # For filtering the data

plottt <- ggplot(summary, aes(x = `shuffle ratio`, y = mean, color = q, linetype=q ,group = q)) +
  geom_line(linetype = "dashed") +
  geom_hline(yintercept = chance_level, linetype = "dotted", color = "red") +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2) +
  labs(y = 'MSE', x = 'Shuffle ratio', color = 'q') +
  theme(
    axis.text = element_text(size = 15),
    axis.title = element_text(size = 15, face = "bold")
  )

print(plottt)
```



Atlanta

```r
m <- 40
tstar <- 20
Num_states <- 50
p <- 0.4
q <- c(0.1, 0.2, 0.35, 0.4, 0.5)
d <- seq(0.05, 1, by = 0.05)
n <- 300
nmc <- 100
```

```r
library(readr)

final_errors <- read_csv("../simulation/simulation_results/final_errors_Atlanta.csv")
final_mse=final_errors[,seq(1,10,2)]
final_sd=final_errors[,seq(2,10,2)]

support = seq(2/m-0.5, (m-1)/m-0.5,by=1/m)
chance_level = sum(support^2/length(support))


library(ggplot2)
library(reshape2)
# Convert matrix to data frame for ggplot2
data_melt <- melt(as.matrix(final_mse))
data_melt_sd <- melt(as.matrix(final_sd))

shuffle_ratio=c(0,d)
data_melt$Var1 <- factor(data_melt$Var1, labels = shuffle_ratio)
data_melt$Var2 <- factor(data_melt$Var2, labels = q)

data_melt_sd$Var1 <- factor(data_melt_sd$Var1, labels = shuffle_ratio)
data_melt_sd$Var2 <- factor(data_melt_sd$Var2, labels = q)


colnames(data_melt)=c('shuffle ratio','q','mean')
colnames(data_melt_sd)=c('shuffle ratio','q','sd')

summary=cbind(data_melt,data_melt_sd$sd)

summary$lower=summary$mean-summary$`data_melt_sd$sd`*1.96
summary$upper=summary$mean+summary$`data_melt_sd$sd`*1.96


library(dplyr)  # For filtering the data

plottt <- ggplot(summary, aes(x = `shuffle ratio`, y = mean, color = q, linetype=q ,group = q)) +
  geom_line(linetype = "dashed") +
  geom_hline(yintercept = chance_level, linetype = "dotted", color = "red") +
  geom_errorbar(aes(ymin = lower, ymax = upper), width = 0.2) +
  labs(y = 'MSE', x = 'Shuffle ratio', color = 'q') +
  theme(
    axis.text = element_text(size = 15),
    axis.title = element_text(size = 15, face = "bold")
  )

print(plottt)
```


MSE
0.15
0.10
0.05
0.00
0 0.05 0.1 0.15 0.2 0.25 0.3 0.35 0.4 0.45 0.5 0.55 0.6 0.65 0.7 0.75 0.8 0.85 0.9 0.95 1
Shuffle ratio
q
0.1
0.2
0.35
0.4
0.5